

CSC633 MACHINE LEARNING



TEXAS A&M UNIVERSITY

Department of Computer
Science & Engineering

CSCE 633: Machine Learning

Optimizing for Neural Networks

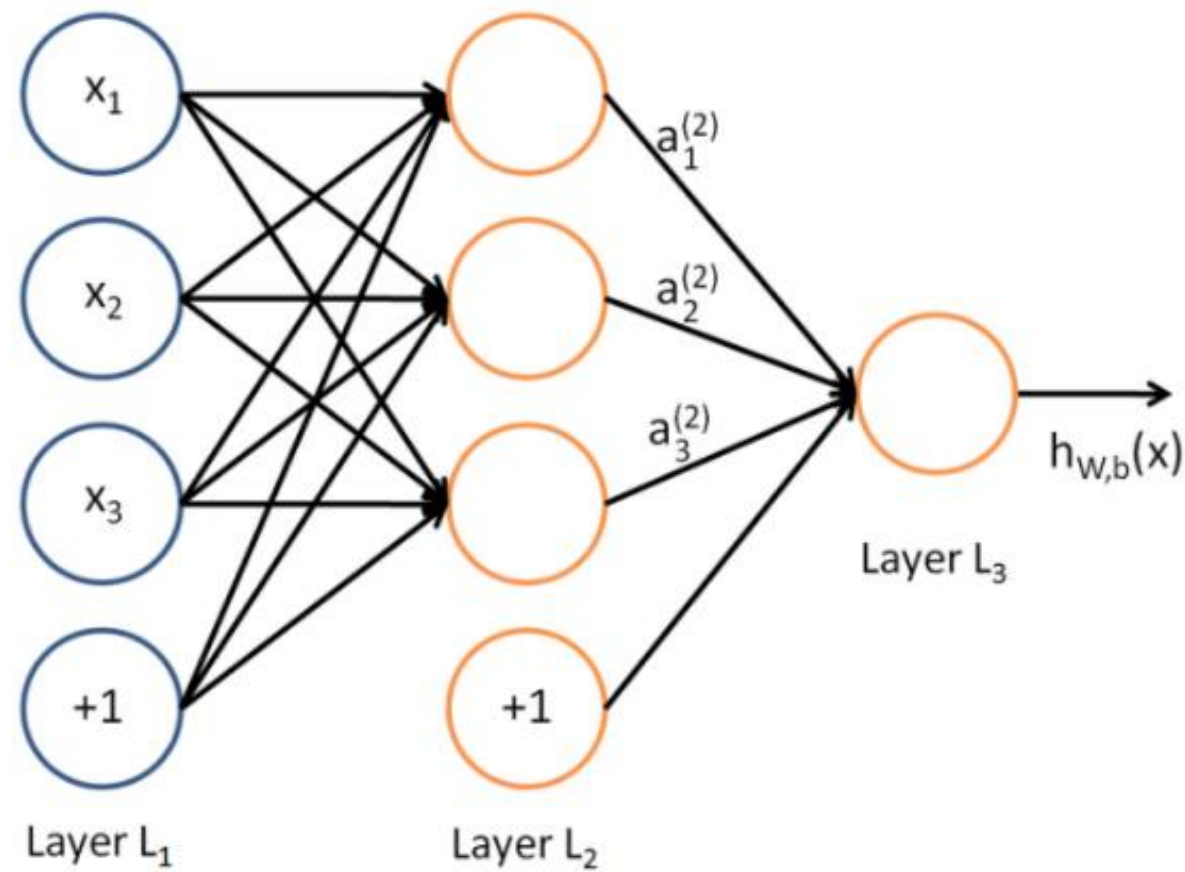
Texas A&M University

Outline

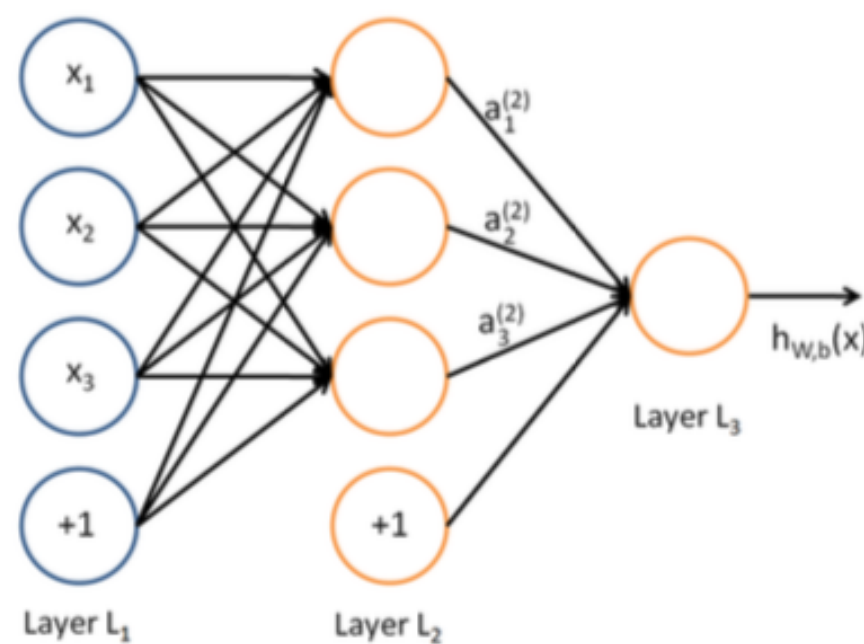
- Activation Function
- Backpropagation
- Optimization
- Neural Network Training and Design

Multilayer Perceptron (1)

- Type of feedforward neural network
- Can model non-linear associations
- “Multi-level combination” of many perceptrons



Multilayer Perceptron: Representation (1)



$$\alpha_1^{(2)} = f(W_{11}^{(1)}x_1 + W_{12}^{(1)}x_2 + W_{13}^{(1)}x_3 + b_1^{(1)})$$

$$\alpha_2^{(2)} = f(W_{21}^{(1)}x_1 + W_{22}^{(1)}x_2 + W_{23}^{(1)}x_3 + b_2^{(1)})$$

$$\alpha_3^{(2)} = f(W_{31}^{(1)}x_1 + W_{32}^{(1)}x_2 + W_{33}^{(1)}x_3 + b_3^{(1)})$$

$$h_{W,b}(\mathbf{x}) = \alpha_1^{(3)} = f(W_{11}^{(2)}\alpha_1^{(2)} + W_{12}^{(2)}\alpha_2^{(2)} + W_{13}^{(2)}\alpha_3^{(2)} + b_1^{(2)})$$

Terminology

$W_{ij}^{(l)}$: connection between unit i in layer l to unit j in layer $l + 1$

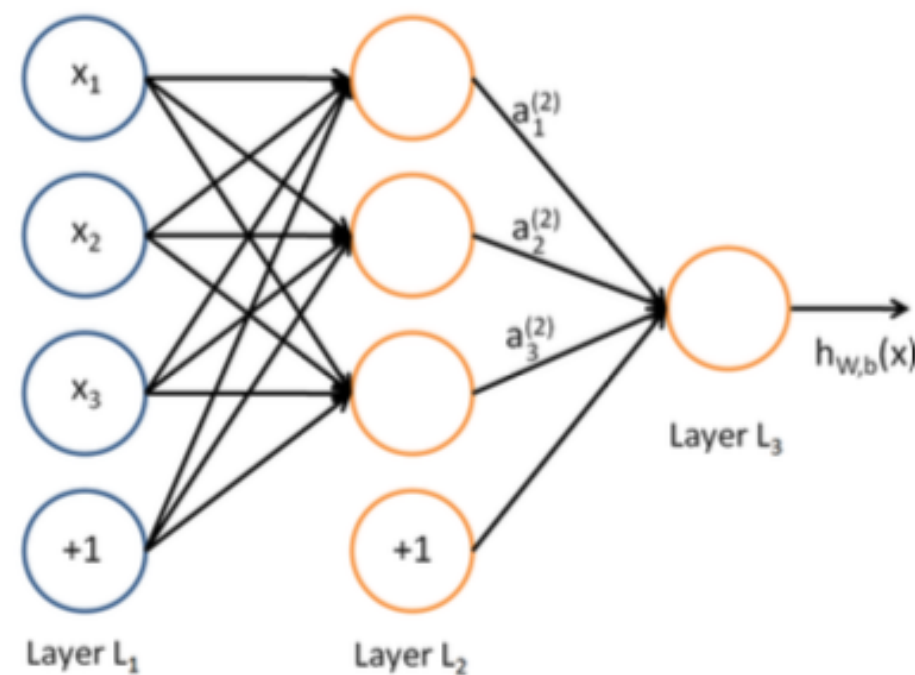
$\alpha_i^{(l)}$: activation of unit i in layer l

$b_i^{(l)}$: bias connected with unit i in layer $l + 1$

Forward propagation: The process of propagating the input to the output through the activation of inputs and hidden units to each node

Multilayer Perceptron: Representation (2)

Matrix notation



$$\alpha^{(2)} = f(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$h_{W,b}(\mathbf{x}) = \alpha^{(3)} = f(\mathbf{W}^{(2)}\alpha^{(2)} + \mathbf{b}^{(2)})$$

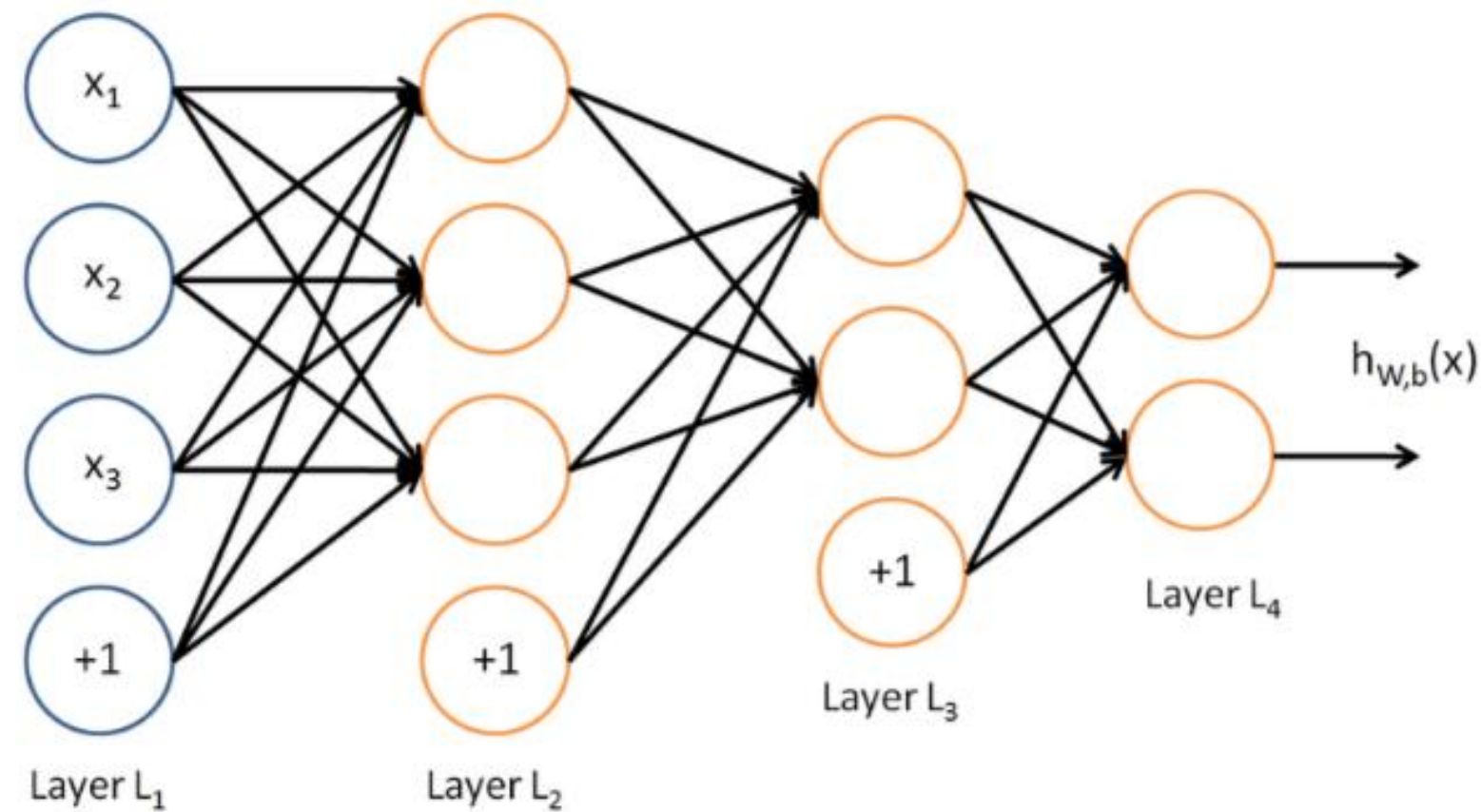
$$\mathbf{W}^{(1)} = \begin{bmatrix} W_{11}^{(1)} & W_{12}^{(1)} & W_{13}^{(1)} \\ W_{21}^{(1)} & W_{22}^{(1)} & W_{23}^{(1)} \\ W_{31}^{(1)} & W_{32}^{(1)} & W_{33}^{(1)} \end{bmatrix}, \mathbf{b}^{(1)} = [b_1^{(1)} \ b_2^{(1)} \ b_3^{(1)}], \text{ etc.}$$

Multilayer Perceptron: Representation (3)

Alternative architectures

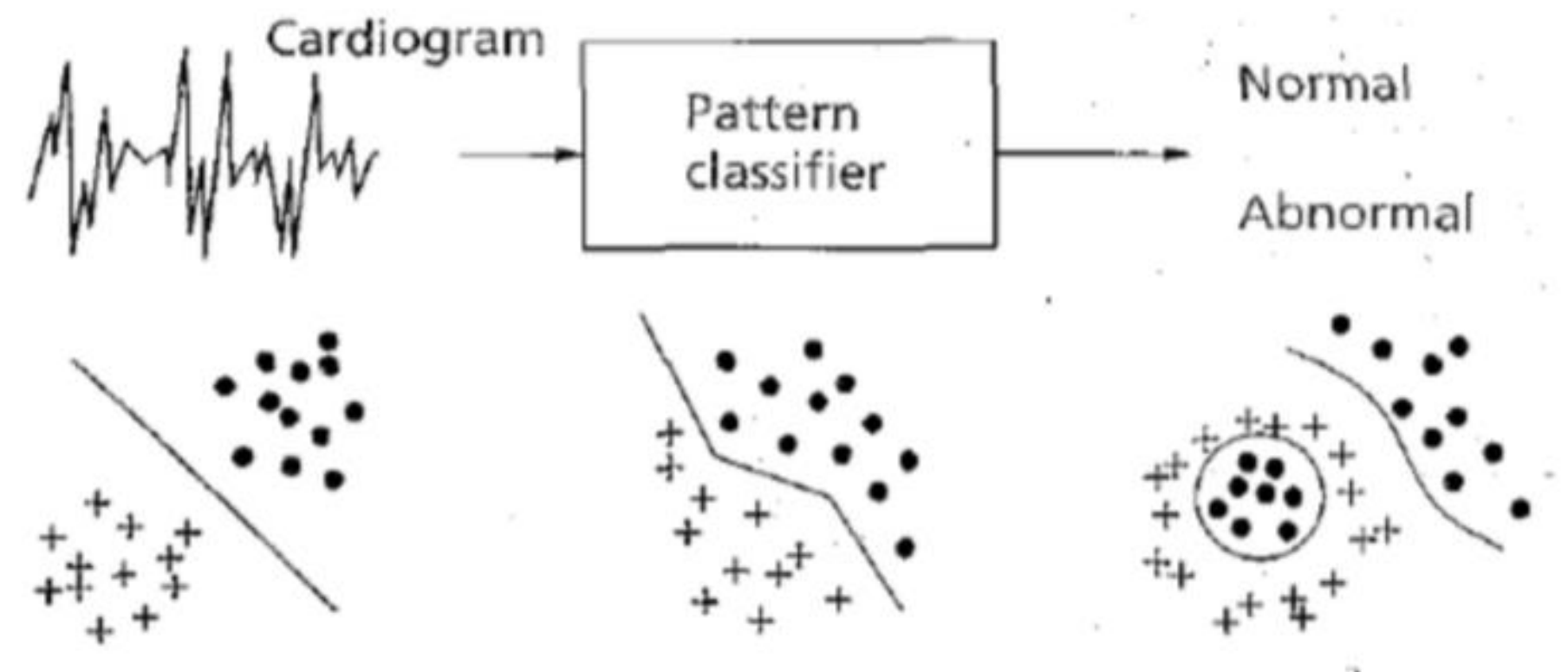
2 hidden layers, multiple output units

e.g. medical diagnosis: different outputs might indicate presence or absence of different diseases



Multilayer Perceptron (2)

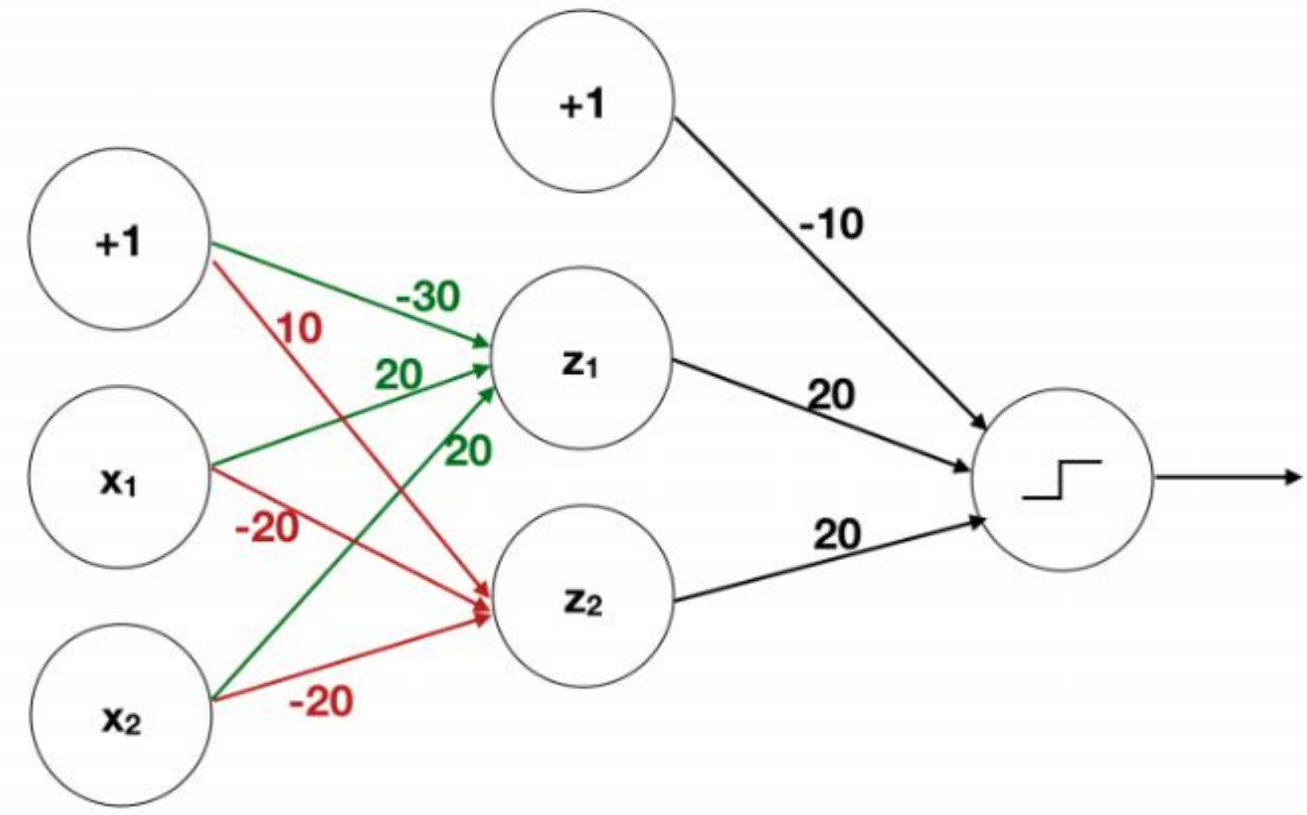
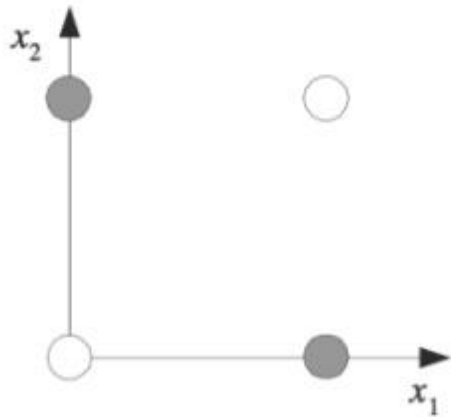
Non-linear feature learning



Multilayer Perceptron: Approximating non-linear functions (1)

Example: Boolean XOR with multilayer perceptrons

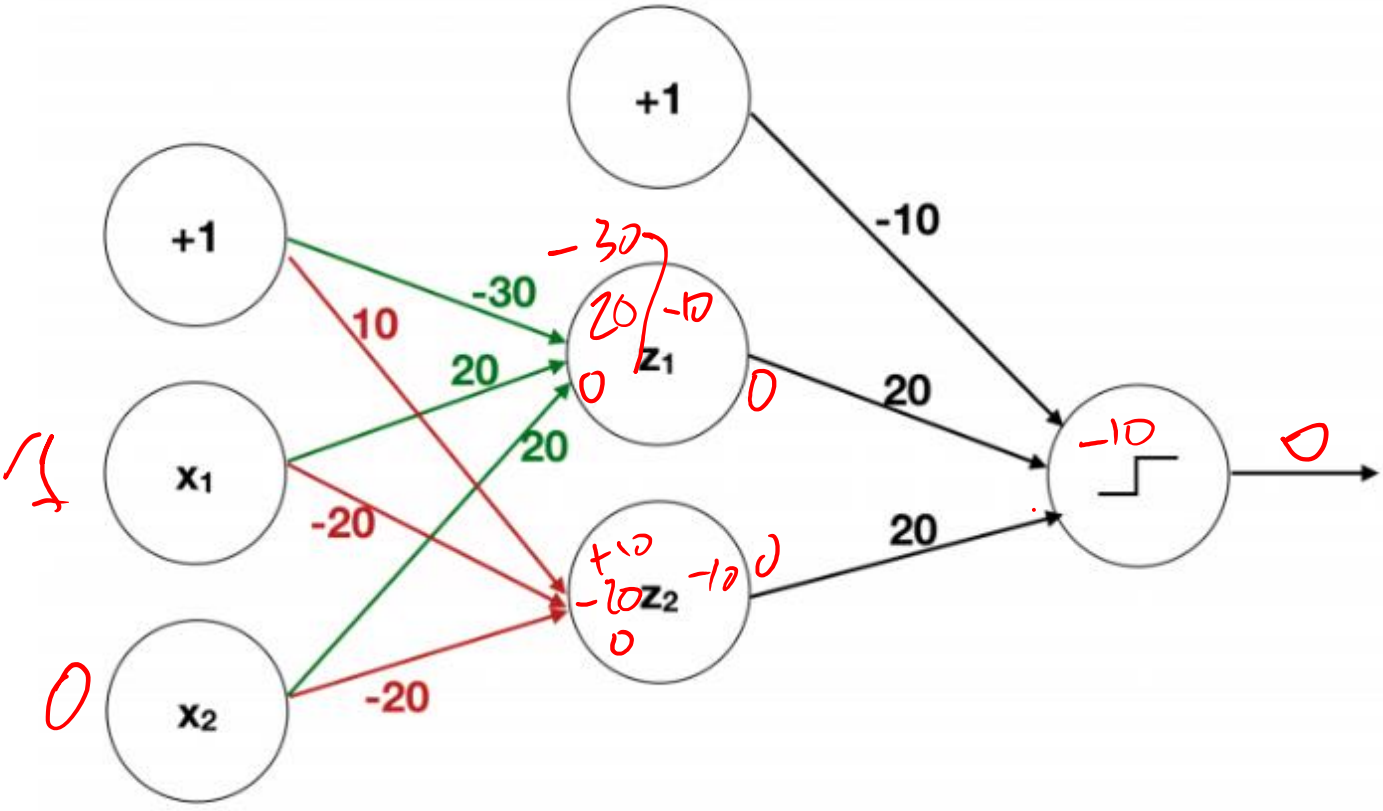
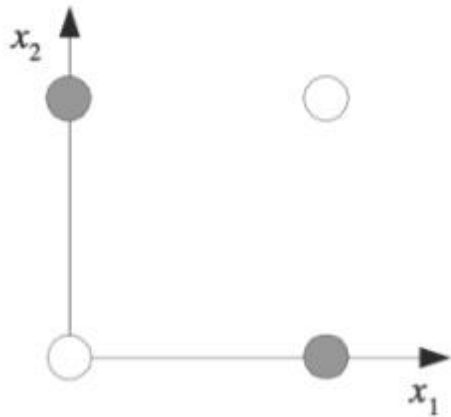
x_1	x_2	z_1	z_2	r
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1



Multilayer Perceptron: Approximating non-linear functions (2)

Example: Boolean **XNOR** multilayer perceptrons

x_1	x_2	z_1	z_2	r
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1



Activation Function (1)

Activation function decides, whether a neuron should be activated or not by calculating weighted sum and further adding bias with it.

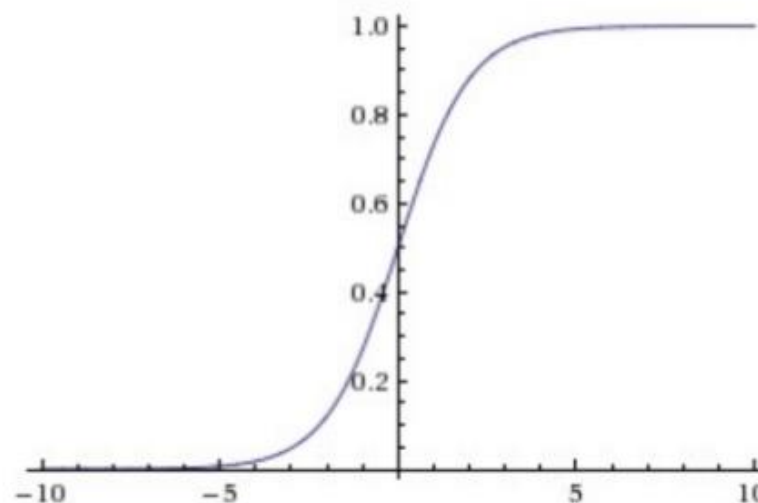
Purpose: introduce non-linearity into the output of a neuron.

- Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$
- Hyperbolic tangent: $s(x) = \tanh(x) = 2\sigma(2x) - 1$
- Rectified Linear Unit (ReLU): $f(x) = \max(0, x)$
- Leaky ReLU: $f(x) = (ax) \cdot \mathbb{I}(x < 0) + (x) \cdot \mathbb{I}(x \geq 0)$ (e.g. $a = 0.01$)

Activation Function (2)

Sigmoid: $s(x) = \frac{1}{1+e^{-x}}$

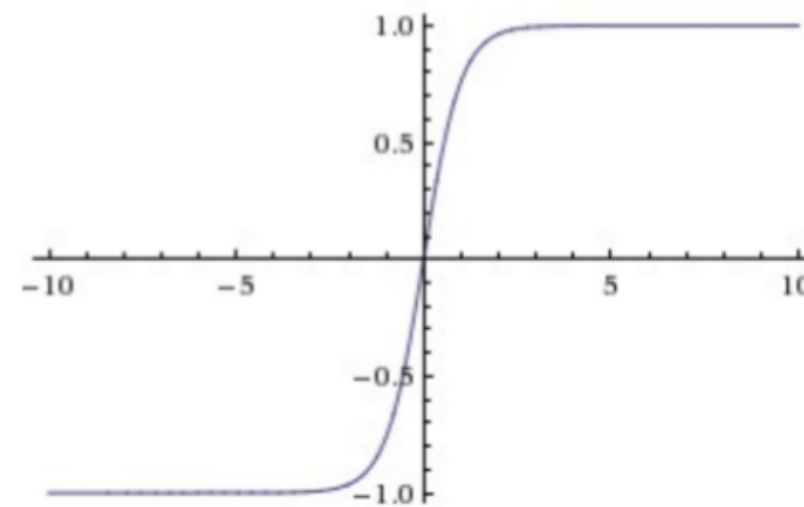
- Transforms a real-valued number between 0 and 1
- Large negative numbers become 0 (not firing at all)
- Large positive numbers become 1 (fully-saturated firing)
- Used historically because of its nice interpretation
- **Saturates gradients:** The gradient at either extremes (0 or 1) is almost zero, “killing” the signal will flow
- **Non-zero centered output:** Can be problematic during training, since it can bias outputs toward being always positive or always negative, causing unnecessary oscillations during the optimization



Activation Function (3)

Hyperbolic tangent: $s(x) = \tanh(x) = 2\sigma(2x) - 1$

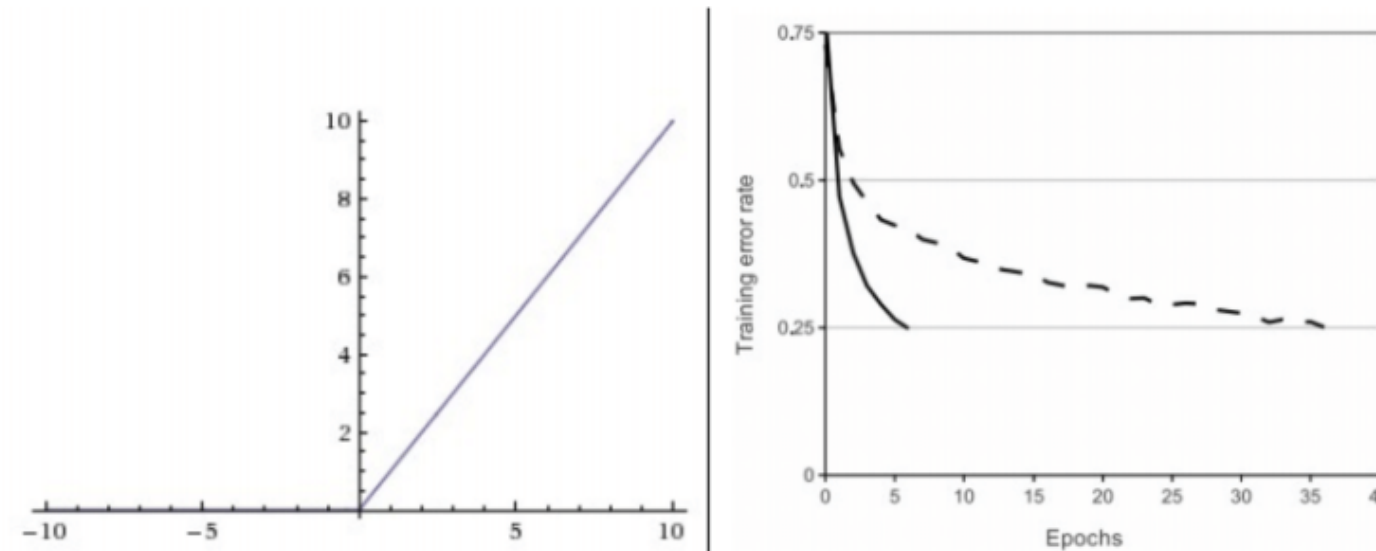
- Scaled version of sigmoid
- Transforms a real-valued number between -1 and 1
- Saturates gradients: Similar to sigmoid
- Output is zero-centered, avoiding some oscillation issues



Activation Function (4)

Rectified Linear Unit (ReLU): $f(x) = \max(0, x)$

- Activation simply thresholded at zero
- Very popular during the last years
- **Accelerates convergence** (e.g. a factor of 6, see bellow) compared to the sigmoid/tanh (due to its linear, non-saturating form)
- **Cheap implementation** by simply thresholding at zero
- Activation can **“die”**: a large gradient flowing through a ReLU neuron could cause the weights to update in such a way that the neuron will never activate on any datapoint again, proper adjustment of learning rate can mitigate that



Activation Function (5)

Leaky ReLU: $f(x) = (ax) \cdot \mathbb{I}(x < 0) + (x) \cdot \mathbb{I}(x \geq 0)$

- Instead of the function being zero when $x < 0$, leaky ReLU will have a small negative slope (e.g. $a = 0.01$)
- Some successful results, but not always consistent

